

Chapter 88

Optical Flow and Stereoscopic Nodes

This chapter covers the numerous stereoscopic and optical flow-based nodes available in Fusion and their related workflows.

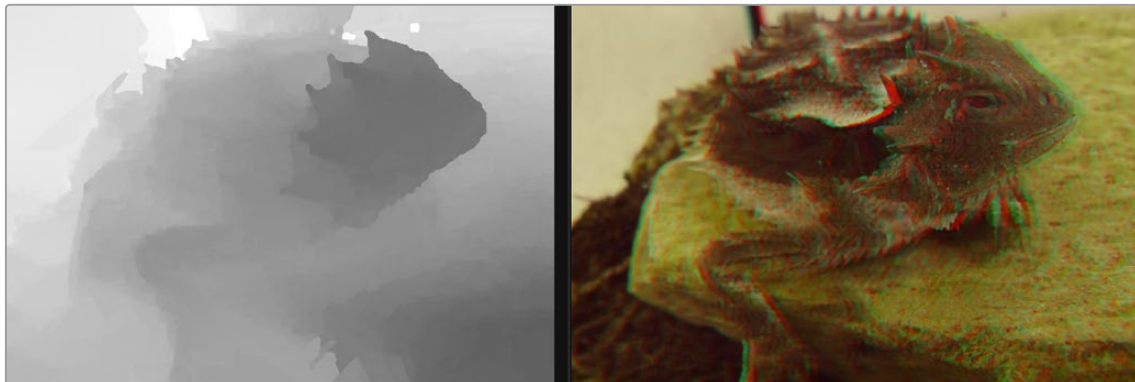
Contents

Overview	1855
Stereoscopic Overview.....	1855
Optical Flow Overview.....	1855
Toolset Overview.....	1856
Working with Aux Deep Channels	1856
Optical Flow Workflows	1857
OpticalFlow.....	1857
TimeSpeed, TimeStretcher.....	1858
SmoothMotion.....	1858
Repair Frame, Tween.....	1858
Advanced Optical Flow Processing.....	1858
Stereoscopic Workflows	1859
Stereo Camera.....	1859
Stereo Materials.....	1860
Disparity.....	1860
NewEye, StereoAlign.....	1860
DisparityToZ, ZToDisparity.....	1861
Separate vs. Stack	1861
Setting Up Stereo in the Node Editor	1861
About the Disparity Channel	1862
Viewing of Disparity and Vector Channels	1862

Stereo and Optical Flow Best Practices	1863
Semi-Transparent Objects	1863
Motion Blur	1863
Depth of Field	1863
Where to Calculate Disparity and Optical Flow?	1863
Cropping the Source	1864
Nodes with Multiple Outputs	1864
Picking from Aux Channels	1864
Vector and Disparity Channels	1864

Overview

Fusion includes 3D stereoscopic and optical flow-based nodes, which can work together or independently of each other to create, repair, and enhance 3D stereoscopic shots.



Stereoscopic comp displayed in the viewers.

Stereoscopic Overview

All stereoscopic features are fully integrated into Fusion's 3D environment. Stereoscopic images can be created using a single camera, which supports eye separation and convergence distance, and a Renderer 3D for the virtual left and right eye. It is also possible to combine two different cameras for a stereo camera rig.

Stereoscopic nodes can be used to solve 3D stereoscopic shooting issues, like 3D rig misalignment, image mirror polarization differences, camera timing sync issues, color alignment, convergence, and eye separation issues. The stereo nodes can also be used for creating depth maps.

Optical Flow Overview

Optical Flow analyzes the motion in a clip and generates motion vectors between neighboring frames. It generates X and Y vectors from the previous frame to the current frame (Back Vectors) and to the next frame in sequence (Forward Vectors). Once calculated, optical flow data can be used by other nodes to create smooth slow motion and variable retiming of clips, repair missing frames, and even correct disparity in stereo 3D clips.

NOTE: The stereoscopic nodes in the Fusion page work independently of the stereoscopic tools in the other DaVinci Resolve pages.

Toolset Overview

Here is an overview of the available nodes.

Optical Flow Nodes

- **Optical Flow > OpticalFlow:** Analyzes motion between neighboring frames in a sequence to generate motion vectors, which can then be used by other nodes for retiming, motion blur, and other effects.
- **Miscellaneous > TimeSpeed:** Retimes a clip at a constant speed using Flow Interpolation mode.
- **Miscellaneous > TimeStretcher:** Retimes a clip at variable speeds using Flow Interpolation mode.
- **Optical Flow > RepairFrame:** Generates a new frame using the motion vectors between two neighboring frames.
- **Optical Flow > SmoothMotion:** Smooths the color or aux channels using motion vectors.
- **Optical Flow > Tween:** Interpolates between two non-sequential images to generate a new frame.
- **Color > CopyAux:** Copies aux channels, including motion vectors, into RGBA more efficiently than Channel Booleans.

Stereoscopic Nodes

- **Stereo > Anaglyph:** Combines stereo images to create a single anaglyph image for viewing.
- **Stereo > Combiner:** sStacks a separate stereo images into a single stacked pair, so they can be processed together.
- **Stereo > Disparity:** Generates disparity between left/right images.
- **Stereo > DisparityToZ:** Converts disparity to Z-depth.
- **Stereo > Global Align:** Shifts each stereo eye manually to do basic alignment of stereo images.
- **Stereo > NewEye:** Replaces left and/or right eye with interpolated eyes.
- **Stereo > Splitter:** Separates a stacked stereo image into to left and right images.
- **Stereo > StereoAlign:** Adjusts vertical alignment, convergence, and eye separation.
- **Stereo > ZToDisparity:** Converts Z-depth to disparity.

Working with Aux Deep Channels

Certain image formats can contain channels other than RGBA color, called aux deep channels. Stereo Disparity and OpticalFlow deal directly with auxiliary deep channels.

Aux channels supported in Fusion include:

- **RGBA:** These are the standard colors.
- **Z:** The eyespace Z coordinate is almost always negative because in eyespace, Fusion's camera sits at (0, 0, 0) looking down the Z-axis. Z values start at Z = 0 at the camera focal point and progressively become more negative for objects deeper in the scene.
- **Coverage:** The percentage of the pixel covered by the frontmost pixel, used for antialiased Z-compositing.

- **Object ID:** These are user-assigned integers to meshes.
- **Material ID:** These are user-assigned integers to materials.
- **Texture Coords:** Normalized texture coordinates stored as (u, v) pairs.
- **Normal Vector:** Normal vector (nx, ny, nz) where the components are typically in the range $[-1, +1]$.
- **Background Color:** The color of the pixel if the frontmost layer were removed, used for antialiased Z-compositing.
- **Vector:** The forward motion vector is an offset (vx, vy) that compares every pixel's position in one frame to the same pixel's position in the next frame.
- **Back Vector:** The backward motion vector is an offset (vx, vy) that compares every pixel's position in one frame to the same pixel's position in the previous frame.
- **World Position:** The position (wx, wy, wz) of the pixel in world coordinates.
- **Disparity:** An offset (dx, dy) that maps a pixel in the Left > Right or Right > Left frames.

Some extra channels are used by specific Fusion nodes.

For example:

- Merge can use the Z channel to perform a depth merge. If the Coverage and BackgroundColor channels are present, it can do a better job on antialiased edges during the Z merge.
- Most image-processing nodes (e.g., BrightnessContrast) have options on their common controls tab to limit their processing by MaterialID and ObjectID.
- The Fog and DepthBlur nodes make use of the Z channel.
- The Texture node makes use of the TexCoord channel.
- The Shader node makes use of the Normal channel.

There are a couple of ways to retrieve or generate those extra channels within Fusion.

For example:

- The Renderer3D node is capable of generating most of these channels.
- The OpticalFlow node generates the Vector and BackVector channels, and then TimeStretcher and TimeSpeed can make use of these channels.
- The Disparity node generates the Disparity channels, and then DisparityToZ, NewEye, and StereoAlign nodes can make use of the Disparity channels.
- The OpenEXR format can be used to import or export aux channels into Fusion by specifying a mapping from EXR attributes to Fusion Aux channels using CopyAux.

Optical Flow Workflows

The Optical Flow analysis is a non real-time process, and depending on your computer, the clip's resolution, and the duration of the clip, it can take some time. Because of this, the general idea is that you pre-generate the motion vectors, either by performing the analysis overnight or using a render farm, and save results into an OpenEXR sequence. The Optical Flow toolset is designed around four types of nodes that either generate, destroy, pass through, or construct the motion vectors.

OpticalFlow

The Optical Flow node generates the Vector and BackVector data. Typically, for optimal performance, you connect the Optical Flow output to a Saver to save the image as OpenEXR files with the motion vectors stored in an aux channel.

TimeSpeed, TimeStretcher

You can create smooth constant or variable slow-motion effects using the TimeSpeed or TimeStretcher nodes. When Optical Flow motion vectors are available in the aux channel of an image, enabling Flow mode in the TimeSpeed or TimeStretcher Interpolation settings will take advantage of the Vector and BackVector channels. For the Flow mode to work, there must be either an upstream OpticalFlow node generating the hidden channels or an OpenEXR Loader bringing these channels in. These nodes use the Vector/BackVector data to do interpolation on the motion channel and then destroy the data on output since the input Vector/BackVector channels are invalid.

For more detail on TimeSpeed or TimeStretcher, see *Chapter 111, “Miscellaneous Nodes,”* in the DaVinci Resolve Reference Manual and Chapter 51 in the Fusion Reference Manual.

SmoothMotion

SmoothMotion can be used to smooth the Vector and BackVector channels or smooth the disparity in a stereo 3D clip. This node passes through, modifies, or generates new aux channels, but does not destroy them.

Repair Frame, Tween

The Tween and Repair Frame nodes are different from standard optical flow nodes because they have the OpticalFlow analysis and motion vector generation built in. Tween will compare two frames and create an in-between frame, which is good for recreating a missing or flawed frame. Repair Frame will look at frames on either side of the current frame and repair scratches, dust marks, and so on. Because these nodes work with flow values between non-sequential frames, they cannot use the optical flow stored in the input image’s Vector/BackVector channels, but rather must regenerate the flow of each frame, do their processing, and then destroy the flow channels. This being the case, these nodes are computationally expensive.

For more detail on Tween or Repair Frame, see *Chapter 112, “Optical Flow,”* in the DaVinci Resolve Reference Manual or Chapter 52 in the Fusion Reference Manual.

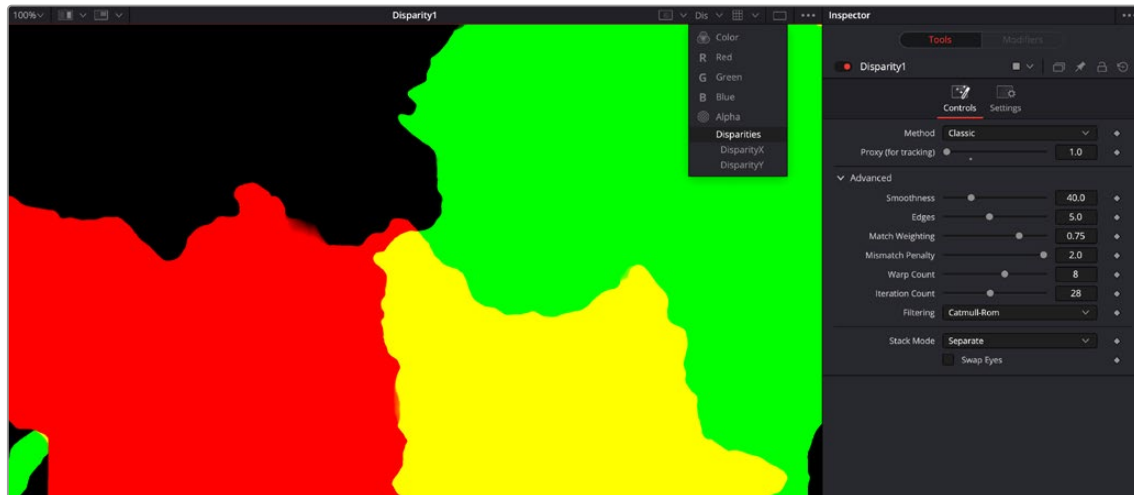
Advanced Optical Flow Processing

The Optical Flow, Repair Frame, and Tween nodes include a faster GPU-based Optical Flow algorithm. When you add the Optical Flow, Repair Frame, or Tween node to a comp, the Inspector includes a Method drop-down menu where you can choose Advanced to enable the GPU-based algorithm. This Advanced method is the same Optical Flow algorithm used in other DaVinci Resolve pages.

By choosing Classic from the Method drop-down menu in the Inspector, you can use the older CPU-based algorithm to maintain compatibility with comps created in previous versions. This method may also be better suited for some Stereo3D processing.

Stereoscopic Workflows

Disparity is the difference between the left and right image. The Disparity map is used by nodes to align and massage the stereo pair of images.



The Disparity node analyzes a stereo pair of images and generates an X&Y disparity map.

The workflow is to load a left and right stereo image pair and process those in the Disparity node. Once the Disparity map is generated, other nodes can process the images.

TIP: When connecting stereo pairs in the node tree, make sure that the left and right images are connected to the left and right inputs of the Disparity node.

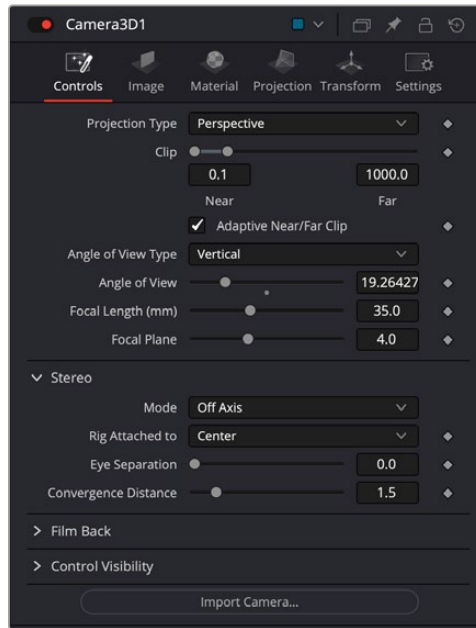
Disparity generation, like Optical Flow, is computationally expensive, so the general idea is that you can pre-generate these channels, either overnight or on a render farm, and save them into an EXR sequence.

The toolset is designed around this philosophy.

Stereo Camera

There are two ways to set up a stereoscopic camera. The common way is to simply add a Camera 3D and adjust the eye separation and convergence distance parameters.

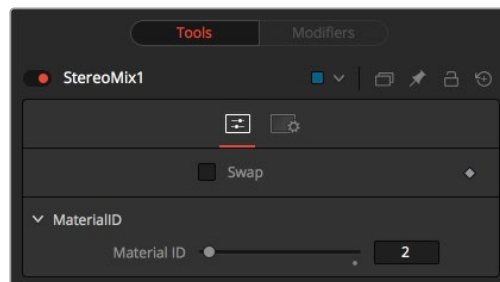
The other way is to connect another camera to the RightStereoCamera input port of the Camera 3D. When viewing the scene through the original camera or rendering, the connected camera is used for creating the right-eye content.



Stereoscopic cameras can be done with a single camera or two connected cameras.

Stereo Materials

Using the Stereo Mix material node, it is possible to assign different textures per eye.



Material viewer showing stereoscopic material.

Disparity

The Disparity node does the heavy lifting of generating disparity maps. This generates the Disparity channel and stores it in the hidden aux channels of their output image.

NewEye, StereoAlign

NewEye and StereoAlign use and destroy the Disparity channel to do interpolation on the color channel.

The hidden channels are destroyed in the process because, after the nodes have been applied, the original Disparity channels would be invalid.

For these nodes to work, there must be either an upstream Disparity node generating the hidden channels or an OpenEXR Loader bringing these channels in.

DisparityToZ, ZToDisparity

These nodes pass through, modify, or generate new aux channels, but do not destroy any.

TIP: If the colors between shots are different, use Color Corrector or Color Curves to do a global alignment first before calculating the Disparity map. Feed the image you will change into the orange input and the reference into the green input. In the Histogram section of the Color Corrector, select Match, and also select Snapshot Match Time. In the Color Curves' Reference section, select Match Reference.

Separate vs. Stack

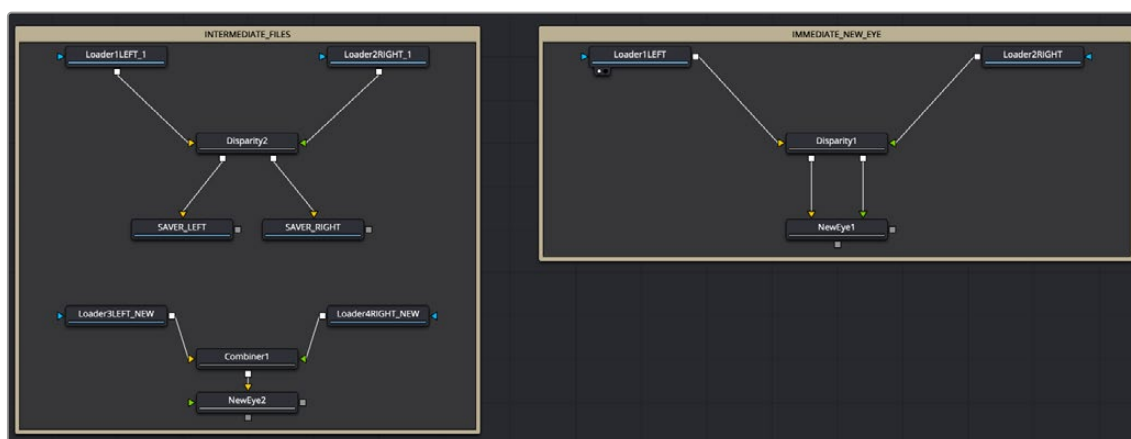
Stereo nodes can work in Separate or Stack modes. When in Stack mode, the left/right eyes are stacked horizontally or vertically, forming one image with double width or height, respectively.

The advantage to using Stack mode is that you do not have to have duplicate branches of the Node Editor for the left and right eyes. As a consequence, you will see Stereo nodes with two inputs and two outputs labeled as "Left" and "Right."

When in Stack mode, the stack should be connected to the left eye input and the Left output should be used for connecting further nodes. In Stack mode, the respective Right eye inputs and outputs are hidden.

Setting Up Stereo in the Node Editor

The disparity generation is the first operation. This can be configured in the Node Editor in two different ways.



Two stereoscopic workflows.

In the above example, the workflow on the right takes the left and right eye, generates the disparity, and then NewEye is used to generate a new eye for the image right away.

The example on the left renders the frames with disparity to intermediate EXR images. These images are then loaded back into Stereo nodes and used to create the NewEye images.

By using Render nodes to compute the disparity first, the later processing of the creative operations can be a much faster and interactive experience.

Although not shown in the above diagram, it is usually a good idea to color correct the right eye to be similar to the left eye before disparity generation, as this helps with the disparity-tracking algorithm. The color matching does not need to be perfect—for example, it can be accomplished using the “Match” option in a Color Corrector’s histogram options.

About the Disparity Channel

The Disparity channel stores the displacement vectors that match pixels in one eye to the other eye. The left image’s Disparity channel will contain vectors that map left>right and the right image’s Disparity channel will contain vectors that map right>left. For example:

```
(xleft, yleft) + (Dleft. x, Dleft. y) -> (xright, yright) (xright, yright) +  
(Dright. x, Dright. y) -> (xleft, yleft)
```

You would expect for non-occluded pixels that $D_{\text{left}} = -D_{\text{right}}$, although, due to the disparity generation algorithm, this is only an approximate equality.

NOTE: Disparity stores both X and Y values because rarely are left/right images perfectly registered in Y, even when taken through a carefully set up camera rig.

Both Disparity and Optical Flow values are stored as un-normalized pixel shifts. In particular, note that this breaks from Fusion’s resolution-independent convention. After much consideration, this convention was chosen so the user wouldn’t have to worry about rescaling the Disparity/Flow values when cropping an image or working out scale factors when importing/exporting these channels to other applications. Because the Flow and Disparity channels store things in pixel shifts, this can cause problems with Proxy and AutoProxy. Fusion follows the convention that, for proxied images, these channels store unscaled pixel shifts valid for the full-sized image. So if you wish to access the Disparity values in a script or via a probe, you need to remember to always scale them by $(\text{image.Width}/\text{image.OriginalWidth}, \text{image.Height}/\text{image.OriginalHeight})$.

Viewing of Disparity and Vector Channels

Aux channels can be displayed directly in the viewers through the Channel viewer button’s menu.

The CopyAux node is used to copy those channels directly into the RGB channels for viewing or further processing. The advantage of using the CopyAux node is that it does static normalization, which reduces a lot of flicker that the viewer’s time-variant normalization causes. When viewing long sequences of aux channels, the CopyAux node has the option to kill off aux channels and keep only the current RGB channels, freeing up valuable memory so you can cache more frames.

TIP: Although you can use the Channel Booleans to copy any aux channel into RGBA, it involves a few additional clicks when compared to CopyAux.

One thing to be aware of is that aux channels tend to consume a lot of memory. A float-32 1080p image containing just RGBA uses about 32 MB of memory, but with all the aux channels enabled it consumes around 200 MB of memory.

Stereo and Optical Flow Best Practices

How you create your composition, the images you are using, and the type of shot you are working on can all have an impact on the success of the Disparity generation and Optical Flow analysis. Below, we'll look at some of the situations to be aware of and how you can avoid some pitfalls when dealing with optical flow.

Semi-Transparent Objects

The Optical Flow and Disparity generation algorithms Fusion uses assume there is only one layer per pixel when tracking pixels from frame to frame. In particular, transparent objects and motion blur will cause problems. For example, a shot flying through the clouds with the semi-transparent clouds in the foreground and a distant landscape background will confuse the Optical Flow/Stereo algorithms, as they do not recognize overlapping objects with different motions. Usually the optical flow will end up tracking regions of one object or the other. If the transparent object and the background are near the same depth and consequently have the same disparity, then it is not a problem.

Motion Blur

Motion blur is also a serious problem for the reason explained in the previous point. The Disparity and Optical Flow algorithms are unsure whether to assign a pixel in the motion blur to the moving object or the background pixel. Because the algorithms used are global in nature, not only the vectors on the motion blur will be wrong, but it will confuse the algorithm on regions close to the motion blur.

Depth of Field

Depth of field is also another problem related to the above two problems. The problem occurs when you have a defocused foreground object over a background object that is moving (Optical Flow case) or shifts between L/R (Stereo Disparity case). The blurred edges will confuse the tracking because they can't figure out that the edges are actually two separate objects.

Where to Calculate Disparity and Optical Flow?

Where you choose to generate optical flow or disparity in your composition can drastically affect the results.

For example, if you have composited a lens flare in, it is better to compute OpticalFlow/Disparity before that, since the semi-transparent lens flare will confuse the tracking algorithms.

If you are color correcting the left/right eyes to match or for deflickering, it is better to apply the OpticalFlow/Disparity afterward, since it will be easier for the tracking algorithm to find matches if the color matches between frames.

If you are removing lens distortion, think carefully about whether you want to do it before or after Disparity computation. If you do it after, your Disparity map will also act as a lens distortion map, combining the two effects as one.

As a general rule of thumb, it is best to use OpticalFlow/Disparity before any compositing operations except an initial color matching correction and a lens distortion removal.

Cropping the Source

As a general tip, if you are cropping down your input images for any reason, it is probably better to compute the optical flow or disparity before the crop and then afterward crop the flow/disparity along with the color.

The reason is that flow/disparity matching works well when there is common pixel data to match in both frames, but when there are pixels that show up in just one frame (or one eye), then the Disparity/OpticalFlow nodes must make a guess and fill in the data. The biggest occlusions going from L $\leftrightarrow$ R are usually pixels along the L/R edges of the images that get moved outside. This is similar for optical flow when you have a moving camera.

Another thing to be aware of are black borders around the edges of your frames, which you should crop away.

Nodes with Multiple Outputs

Many of the stereo nodes in the Fusion toolset have multiple outputs. This can cause some confusion to new users. One particularly confusing thing is that when you drag a Stereo node to the view, it will always display the left output. There is no way to view the right output without connecting another node like BC (BrightnessContrast) to the right output and viewing that.

Picking from Aux Channels

Some nodes, like StereoAlign, allow one to drag pick from the Z or Disparity auxiliary channels. You must pick from a node upstream of the StereoAlign, not from the output of the StereoAlign. If you try to pick a disparity from the output of a StereoAlign node, you will get nothing because StereoAlign consumes/destroys the Disparity aux channel (and even if it did not destroy the Disparity channel, you would still be picking the wrong value since you would be picking from the aligned result).

The typical workflow for picking is:

- 1 View StereoAlign in the left view.
- 2 View the node upstream of StereoAlign in the right view.
- 3 Pick the Disparity value from the left eye in the right view.

Although this picking functionality does not operate any differently from normal picking of color channels, this issue may cause some confusion. If it helps, the analogous workflow mistake with color nodes would be a user trying to pick a gradient color for a Background node from a view showing the Background node itself (you are trying to pick a color for a node from its own output).

Another issue that you need to be aware of is which eye you are picking. To avoid problems, it's a good idea to always pick from the left eye. The reason is that the Disparity channels for the left and right eyes are different, and when you pick from a horizontal/vertical stereo stack, Fusion has no way of knowing whether you picked the Disparity value from the left or right eye.

The above are not hard and fast rules; rather, they are guidelines to prevent foot shootings. If you understood the above reasoning fully, you'll realize there are exceptions, like picking disparity from the left output of DisparityToZ and Z from the left/right output of ZToDisparity, where everything is okay.

Vector and Disparity Channels

The Vector and BackVector channels store the forward and reverse optical flow.

The Vector channel might be better named "forward vector" or "forward flow," since the name "Vector" to describe a channel is "not technically correct," as the more mathematically-inclined user might recognize that all the channels except the scalar channels Z/ID are technically "vector" channels.

A frames Vector aux channel will store the flow forward from the current frame to the next frame in the sequence, and the BackVector aux channel will store the flow backward from the current frame to the previous frame. If either the previous or next frames do not exist (either not on disk or the global range of a Loader does not allow OpticalFlow to access them), Fusion will fill the corresponding channels with zeros (transparent black).

The Disparity channel stores the displacement vectors that match pixels in one eye to the other eye. The left image's Disparity channel will contain vectors that map left > right and the right image's Disparity channel will contain vectors that map right > left.

For example:

```
(xleft, yleft) + (Dleft. x, Dleft. y) -> (xright, yright)
(xright, yright) + (Dright. x, Dright. y) -> (xleft, yleft)
```

You would expect for non-occluded pixels that $D_{\text{left}} = -D_{\text{right}}$, although due to the disparity generation algorithm, this is only an approximate equality. Note that Disparity stores both X and Y values because rarely are left/right images perfectly registered in Y, even when taken through a carefully set up camera rig.

Disparity and Optical Flow values are stored as un-normalized pixel shifts. In particular, note that this breaks from Fusion's resolution-independent convention. After much consideration, this convention was chosen so the user wouldn't have to worry about rescaling the Disparity/Flow values when cropping an image or working out scale factors when importing/exporting these channels to other applications. Because the Flow and Disparity channels store things in pixel shifts, this can cause problems with Proxy and AutoProxy. The convention that Fusion follows is that, for proxied images, these channels store unscaled pixel shifts valid for the full-sized image. So if you wish to access the disparity values in a script or via a probe, you need to remember to always scale them by $(\text{image. Width}/\text{image. OriginalWidth}, \text{image. Height}/\text{image. OriginalHeight})$.

When using Vector and BackVector aux channels, remember that all nodes expect these aux channels to be filled with the flow between sequential frames.

More precisely, if you have sequence of three frames A, B, C, then:

B	Vector will contain the flow B>C
B	BackVector will contain the flow B>A
A	Vector will contain the flow A>B
A	BackVector is written with zeros as there is no frame before A
C	Vector is written with zeros as there is no frame D to flow C>D
C	BackVector will contain the flow C>B

When working with these channels, it is the user's responsibility to follow these rules (or for clever users to abandon them). Nodes like TimeStretcher will not function correctly since they still expect the channels to contain flow forward/back by 1 frame.

NOTE: Currently DoD/Rol is not supported for all Fusion nodes.



Fusion Page Effects

CONTENTS

89	3D Nodes	1867	107	LUT Nodes	2377
90	3D Light Nodes	1985	108	Mask Nodes	2386
91	3D Material Nodes	2004	109	Matte Nodes	2427
92	3D Texture Nodes	2033	110	Metadata Nodes	2498
93	Blur Nodes	2058	111	Miscellaneous Nodes	2506
94	Color Nodes	2083	112	Optical Flow	2545
95	Composite Nodes	2158	113	Paint Node	2565
96	Deep Image Nodes	2178	114	Particle Nodes	2575
97	Deep Pixel Nodes	2197	115	Position Nodes	2635
98	Effect Nodes	2213	116	Resolve Connect	2654
99	Film Nodes	2250	117	Shape Nodes	2660
100	Filter Nodes	2269	118	Stereo Nodes	2700
101	Flow Nodes	2286	119	Tracking Nodes	2732
102	Flow Organizational Nodes	2289	120	Transform Nodes	2788
103	Fuses	2294	121	USD Nodes	2816
104	Generator Nodes	2296	122	VR Nodes	2869
105	I/O Nodes	2336	123	Warp Nodes	2880
106	Layer Nodes	2363	124	Modifiers	2910